Week 1 Lab: DC Motor, PWM, and Encoder Measurement

ENGR 3303 – Controls Engineering
Fall 2026

Main goal. By the end of these two lab periods, you will be able to wire and operate a DC gearmotor through an L298N motor driver, control its direction and speed using Arduino PWM, read the motor encoder, and collect the first open-loop motor-speed data for this course.

Hardware Used

Each team will use:

- Arduino Uno or compatible board;
- one **12 V, 200 RPM encoder gearmotor** with 65 mm wheel;
- one **L298N dual H-bridge motor-driver module**;
- instructor-provided 12 V motor power supply;
- USB cable for the Arduino;
- Dupont jumper wires;
- laptop with Arduino IDE.

Important safety rules

- Turn **motor power OFF** before changing any wiring.
- Power the Arduino from USB. Do **not** use the L298N 5 V terminal to power the Arduino in this lab.
- The Arduino GND and L298N GND **must be connected together**.
- Keep fingers, wires, clothing, and tools away from the spinning wheel.
- Never intentionally stall the motor.
- Set PWM to zero before reversing direction.
- Do not connect the encoder to 12 V.

Pin Assignment for This Course

Use the following pin assignment unless your instructor tells you otherwise.

Function	Arduino pin	Connection
Motor PWM	D9	L298N ENA
Motor direction 1	D7	L298N IN1
Motor direction 2	D8	L298N IN2
Encoder channel A	D2	Encoder A
Encoder channel B	D3	Encoder B
Common ground	GND	L298N GND and encoder GND
Encoder power	5 V	Encoder VCC, only after instructor verification

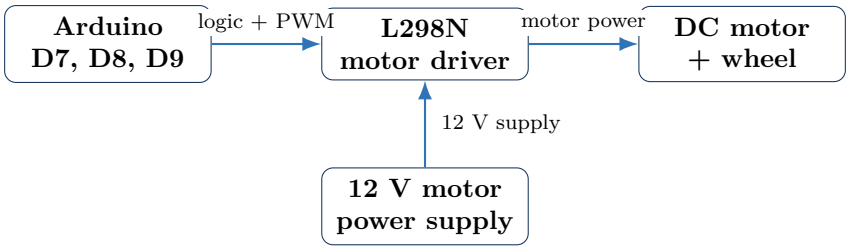
Encoder-wire rule. Do not guess the wire colors. The encoder leads must be identified or labeled by the instructor as `VCC`, `GND`, `A`, and `B`. The two motor-power leads connect to the L298N output terminals.

Lab 1A – Motor Driver and PWM

Suggested time: 75 minutes

Part 1 – Understand the Signal Flow

The Arduino cannot drive this motor directly. The Arduino sends low-power logic commands to the L298N, and the L298N switches the higher-power motor supply.



Part 2 – Wire the Motor Driver

Keep the 12 V motor supply OFF during this entire step.

1. Connect the two motor-power wires to **OUT1** and **OUT2** on the L298N.
2. Connect Arduino **D7** to L298N **IN1**.

3. Connect Arduino **D8** to L298N **IN2**.
4. Remove the small **ENA jumper** so Arduino PWM can control ENA.
5. Connect Arduino **D9** to L298N **ENA**.
6. Connect Arduino **GND** to L298N **GND**.
7. Connect the external motor supply positive terminal to L298N **12V/+V**.
8. Connect the external motor supply negative terminal to L298N **GND**.
9. Leave the L298N **5 V terminal unconnected**.
10. Have your instructor check the wiring **before applying motor power**.

> **Instructor checkpoint 1:** Wiring inspected before 12 V motor power is turned on.

Part 3 – First Motor Test

Open Arduino IDE and upload the following program.

```
const int ENA = 9;
const int IN1 = 7;
const int IN2 = 8;

void setup() {
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  // Start safely with motor off
  analogWrite(ENA, 0);

  // Forward direction
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);

  delay(1000);

  // Moderate speed
  analogWrite(ENA, 120);
}

void loop() {
}
```

After uploading:

1. Make sure the wheel is free to rotate.
2. Turn on the 12 V motor supply.

3. Observe the motor.
4. If the wheel rotates in the opposite physical direction from the direction you want to call "forward," turn off motor power and swap OUT1/OUT2, or redefine the direction in code.
5. Set PWM back to zero before editing or rewiring.

Part 4 – Test PWM Values

Arduino `analogWrite()` uses values from 0 to 255.

Modify this line:

```
analogWrite(ENA, 120);
```

Test the following PWM values one at a time:

$$80, \quad 120, \quad 160, \quad 200, \quad 255.$$

For each value:

1. set the PWM value;
2. upload the program;
3. observe whether the motor starts smoothly;
4. listen for unusual noise;
5. record a qualitative observation.

PWM	Motor starts?	Observation
80		
120		
160		
200		
255		

Part 5 – Reverse Direction Safely

To reverse the motor:

1. first command `analogWrite(ENA, 0);`;
2. wait about 0.5 s;
3. change the direction pins to

```
analogWrite(ENA, 0);
delay(500);

digitalWrite(IN1, LOW);
digitalWrite(IN2, HIGH);

analogWrite(ENA, 120);
```

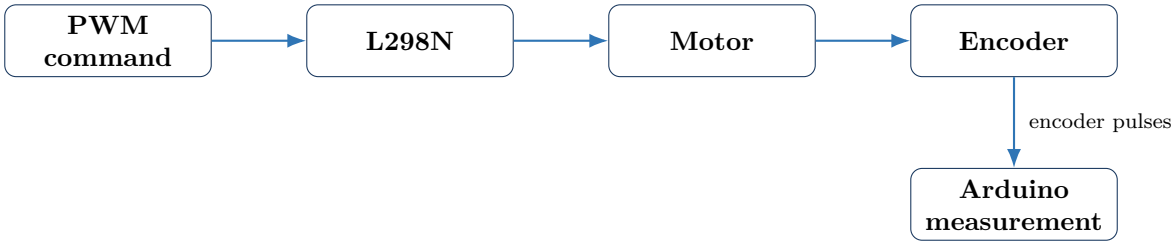
> **Lab 1A checkoff** Demonstrate to the instructor: (1) motor OFF, (2) forward rotation, (3) at least two different PWM speeds, and (4) safe reversal.

Lab 1B – Encoder Measurement and Open-Loop Characterization

Suggested time: 75 minutes

Part 1 – What the Encoder Does

The motor encoder provides pulses as the motor rotates. Instead of only observing the wheel, Arduino can count those pulses and use them as a speed measurement.



Part 2 – Connect the Encoder

Turn OFF the 12 V motor supply before adding encoder wiring.

Use the instructor-labeled encoder leads:

1. Encoder **VCC** → Arduino **5 V**.
2. Encoder **GND** → Arduino **GND**.
3. Encoder **A** → Arduino **D2**.
4. Encoder **B** → Arduino **D3**.
5. Keep Arduino GND connected to L298N GND.
6. Ask the instructor to check the encoder wiring.

> **Instructor checkpoint 2:** Encoder pinout and supply voltage verified before power is applied.

Part 3 – Count Encoder Pulses

Upload this code.

```
const int ENA = 9;
const int IN1 = 7;
const int IN2 = 8;

const int ENC_A = 2;
const int ENC_B = 3;

volatile unsigned long encoderCount = 0;

unsigned long lastPrint = 0;
const unsigned long sampleTime = 1000; // ms
```

```
12
13 void countPulse() {
14   encoderCount++;
15 }
16
17 void setup() {
18   Serial.begin(115200);
19
20   pinMode(ENA, OUTPUT);
21   pinMode(IN1, OUTPUT);
22   pinMode(IN2, OUTPUT);
23
24   pinMode(ENC_A, INPUT_PULLUP);
25   pinMode(ENC_B, INPUT_PULLUP);
26
27   attachInterrupt(digitalPinToInterrupt(ENC_A),
28                   countPulse, RISING);
29
30   digitalWrite(IN1, HIGH);
31   digitalWrite(IN2, LOW);
32   analogWrite(ENA, 120);
33
34   lastPrint = millis();
35 }
36
37 void loop() {
38   unsigned long now = millis();
39
40   if (now - lastPrint >= sampleTime) {
41     noInterrupts();
42     unsigned long counts = encoderCount;
43     encoderCount = 0;
44     interrupts();
45
46     float dt = (now - lastPrint) / 1000.0;
47     float countsPerSecond = counts / dt;
48
49     Serial.print("Encoder counts/s = ");
50     Serial.println(countsPerSecond);
51
52     lastPrint = now;
53   }
54 }
```

Open **Tools** → **Serial Monitor** and select **115200 baud**.
You should see a new encoder count-rate value approximately once per second.

> If the motor rotates but the count remains zero, turn motor power OFF and check encoder VCC, GND, Channel A, and the D2 connection.

Part 4 – Collect Open-Loop Speed Data

Keep the motor direction the same. For each PWM value below:

1. change the PWM command;
2. wait 3–5 seconds for the motor to settle;
3. record three consecutive encoder count-rate readings;
4. calculate the average.

PWM	Reading 1	Reading 2	Reading 3	Average counts/s
80				
120				
160				
200				
255				

Part 5 – Make a PWM-vs-Speed Plot

Use Python:

```
import numpy as np
import matplotlib.pyplot as plt

pwm = np.array([80, 120, 160, 200, 255])

# Replace the zeros with your measured averages.
speed = np.array([0, 0, 0, 0, 0])

plt.plot(pwm, speed, "o-", linewidth=1.5, markersize=7)
plt.xlabel("PWM command")
plt.ylabel("Encoder counts per second")
plt.title("Open-Loop DC Motor Characterization")
plt.grid(True)
plt.xlim([70, 265])
plt.show()
```

Include the plot in your lab submission.

Part 6 – Optional RPM Conversion

Your encoder produces a certain number of counted Channel-A pulses per output-shaft revolution. This value is called the **counts per revolution (CPR)** for the counting method used in this lab.

For the course motor, the output shaft was experimentally calibrated using Encoder A with `RISING` edges only. The measured course value is

$$\boxed{N_{\text{CPR}} = 494.0 \text{ counts/output revolution}}.$$

Convert count rate to RPM using

$$\boxed{\text{RPM} = \frac{\text{counts/s}}{494.0} \times 60}$$

Use this value only with the same counting method. If the interrupt mode or encoder-counting method changes, recalibrate the CPR.

Questions to Submit

Answer briefly.

1. What is the role of the L298N motor driver? Why should the motor not be connected directly to an Arduino output pin?
2. What happens to motor speed as PWM increases?
3. Does doubling PWM exactly double the measured motor speed? Use your data.
4. At what PWM value did your motor reliably begin rotating?
5. What does the encoder add that you did not have in Lab 1A?
6. Is this experiment open-loop or closed-loop? Explain.
7. Suppose PWM = 160 gives your desired speed with no load. Why might the same PWM fail to maintain that speed when the load changes?
8. Why will feedback be useful later in this course?

What to Submit

One submission per team:

- completed PWM observation table from Lab 1A;
- completed encoder-data table from Lab 1B;
- PWM-vs-speed plot;
- answers to Questions 1–8;
- your final Arduino code.

End-of-Lab Shutdown

Before leaving:

1. set PWM to zero;
2. turn OFF the 12 V motor supply;
3. disconnect USB only after the motor supply is off;

4. leave the wiring organized for instructor inspection;
5. return all components to the assigned kit.

Hardware reference: The L298 is a dual full-bridge driver intended for inductive loads including DC motors. For the supplied course motor, use the instructor-labeled encoder pinout and the experimentally calibrated value $N_{\mathrm{CPR}} = 494.0$ for Encoder A with `RISING` edges only.